Comprehensive List Generation for Multi-Generator Reranking

Hailan Yang
Kuaishou Technology
Beijing, China
yanghailan@kuaishou.com

Zhenyu Qi
Kuaishou Technology
Beijing, China
qizhenyu@kuaishou.com

Shuchang Liu*
Kuaishou Technology
Beijing, China
liushuchang@kuaishou.com

Xiaoyu Yang
Kuaishou Technology
Beijing, China
yangxiaoyu@kuaishou.com

Xiaobei Wang
Kuaishou Technology
Beijing, China
wangxiaobei03@kuaishou.com

Xiang Li
Kuaishou Technology
Beijing, China
lixiang44@kuaishou.com

Lantao Hu
Kuaishou Technology
Beijing, China
hulantao@kuaishou.com

Han Li
Kuaishou Technology
Beijing, China
lihan08@kuaishou.com

Kun Gai
Unaffiliated
Beijing, China
gai.kun@qq.com

Abstract

Reranking models solve the final recommendation lists that best fulfill users' demands. While existing solutions focus on finding parametric models that approximate optimal policies, recent approaches find that it is better to generate multiple lists to compete for a "pass" ticket from an evaluator, where the evaluator serves as the supervisor who accurately estimates the performance of the candidate lists. In this work, we show that we can achieve a more efficient and effective list proposal with a multi-generator framework and provide empirical evidence on two public datasets and online A/B tests. More importantly, we verify that the effectiveness of a generator is closely related to how much it complements the views of other generators with sufficiently different rerankings, which derives the metric of list comprehensiveness. With this intuition, we design an automatic complementary generator-finding framework that learns a policy that simultaneously aligns the users' preferences and maximizes the list comprehensiveness metric. The experimental results indicate that the proposed framework can further improve the multi-generator reranking performance.

CCS Concepts

• **Information systems** → **Learning to rank**; *Information retrieval diversity*; • **Computing methodologies** → *Ensemble methods*.

Keywords

Learning to rank; generative models; recommendation diversity

*Corresponding author

SIGIR '25, July 13–18, 2025, Padua, Italy

ACM ISBN 979-8-4007-1592-1/2025/07
https://doi.org/10.1145/3726302.3729933

ACM Reference Format:
Hailan Yang, Zhenyu Qi, Shuchang Liu, Xiaoyu Yang, Xiaobei Wang, Xiang Li, Lantao Hu, Han Li, and Kun Gai. 2025. Comprehensive List Generation for Multi-Generator Reranking. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '25), July 13–18, 2025, Padua, Italy.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3726302.3729933

1 Introduction

Recommender systems filter information of interest to users for a wide range of web services such as e-commerce, news, social media, and video sharing platforms. Practical recommendation settings usually involve large and dynamic candidate sets, which bring forth the challenge of accuracy-latency trade-off: sophisticated solutions cannot meet the latency demand due to the large candidate set, and simple models cannot meet the accuracy demand that directly influences user satisfaction. As a countermeasure, the multi-stage design [35, 44, 54] has been proposed to iteratively refine and filter relevant items/contents that best align with the user's interests. In the final reranking stage [31] that determines the final exposure, the list provided from previous ranking stages is reordered before presentation to the user, considering the cross-item influence. For example, an item with repeating information to a previous item may not satisfy the user's need for content diversity, but an item may become more attractive if the user has previously interacted with a complementary item.

Formally, the reranking task aims to find the optimal list that maximizes the user response taking into account the cross-item mutual influence. Early studies [6, 56] found that the mutual influence is closely related to the recommendation diversity and have proposed several heuristic-based techniques to balance the diversity-accuracy trade-off. Later approaches [1, 17, 34], denoted as the generator-only paradigm, adopted machine learning methods that learn a policy model that directly models the interactions among the candidates, learns how different rankings affect the list-level reward, and generates the ranking according to the optimum reward. However, finding the best ranking is combinatorially complex, and the problem space grows exponentially with the size of the candidate set and the size of the output recommendation list.

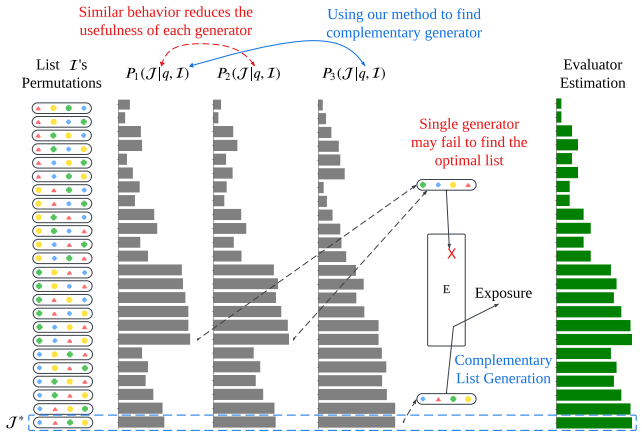


Figure 1: Intuitive example for the benefit of multi-generator design in reranking task and the motivation of list comprehensiveness. P_1, P_2, and P_3 are different generators. E represents the efficient and accurate list reward estimator that select the best list as exposure. $\mathcal{I}$ and $\mathcal{J}^*$ corresponds to the initial list and the optimal list. q denotes the user request.

Furthermore, it is unknown how the user would respond to an ordering different from the observed data sample, which drives the efforts to find extra guidance as reinforcement. To mitigate these issues, several recent works [39, 41] have proposed the two-step generator-evaluator (G-E) framework, where the generator selects a number of high-value candidate lists among all possible permutations, avoiding the exhaustive search in the list space; then the evaluator accurately estimates the utility/reward of each list, serves as a simulator that mimics the user's response for unseen lists, and picks the list with the maximum reward as the recommendation. Yet, the naive list permutation process in the generator step of the G-E framework injects random noises, which do not guarantee accuracy during list exploration. Furthermore, the single-generator design may still be ineffective even with the similarity-based exploration [41] when the optimal list is outside of the local optima (e.g., The optimal region of P_1 in Figure 1 only covers the lists around the sub-optimal list in the ground truth provided by the evaluator).

In this work, we investigate an alternative solution to the reranking task, which overcomes the aforementioned limitations with a **multi-generator-evaluator (MG-E)** framework. Specifically, multiple generators are included to collaboratively produce valuable and diverse candidate lists before feeding the evaluator. On the one hand, the combined generators form a list-level ensemble model which can easily avoid the potential local optimum of one single generator, offering more high-quality choices for the evaluator. Consequently, the overall accuracy is theoretically and empirically superior to the single-generator methods. On the other hand, restricting the list generation opportunities to the selected generators significantly reduces the number of lists to evaluate in the next evaluator step, compared to the permutation-based solutions.

Despite these advantages, the independent generators may have a chance of generating rankings that are too similar in practice (see P_1 and P_2 in Figure 1 for example), especially when they are pre-trained with the dataset under the same user environment . This lack of list-level diversity would potentially reduce the effectiveness of the multi-generator design and we denote this problem as the **List Comprehensiveness Challenge**. Note that this challenge does not exist in the permutation-based list exploration in the G-E framework, but rather a unique challenge under the MG-E framework. To address this challenge, we further enhance the MG-E framework with a **List Comprehensiveness(LC) metric** that evaluates how different the generated lists are and guides the selection of generators. We present an intuitive example in Figure 1, and we can see that pairing the P_1 generator with P_3 would help increase the chance of finding the optimal list due to their diverse behaviors, while pairing P_1 with P_2 fails to achieve this goal. Without loss of generality, all generators may have their own local optimal list covering one side of the ground-truth distribution, but we need a method to ensure their differences, which would benefit the ensemble effect of the generators. We provide an empirical analysis in section 3.3 on how this metric correlates with the final reranking performance through different combinations of generators.

In addition, there are still limitations of manually designed generator combinations even with the LC metric as guidance: 1) The generator selection space grows combinatorially with M, making it increasingly difficult to analyze; 2) And there might still exist certain local optima patterns that no existing reranking solution can effectively cover. Using the same example in Figure 1, both P_2 and P_3 align with user preference, but P_3 has better LC performance with P_1 as the existing generator, thus the framework will automatically learn a generator that is close to P_3. As a result, we derive a simple but effective learning framework that automatically finds a generator that complements the existing generators (See details in section 3.4). During training, the corresponding model maximizes the gain of the overall LC metric, as well as the alignment with the user preference. To illustrate the generalizability of the learning algorithm, we derive corresponding detailed solutions for both autoregressive and non-autoregressive list generation processes. We denote this general extension technique as **Complementary LIst Generation (CLIG)** and verify its effectiveness in improving the ensemble effect through offline experiments in two public datasets, as well as online A/B test in our industrial recommender system.

In the remainder of this paper, we will present the detailed support for the following contribution:

- We prove that the MG-E framework is a superior design over generator-only solutions and perturbation-based G-E solutions.
- We identify the list comprehensiveness challenge in the MG-E framework and explain its relationship with the list-level ensemble effect through the LC metric.
- We propose the CLIG extension technique that can further boost the reranking performance and verify the sub-optimality of single-generator solutions and manually designed MG-E solutions.

2 Related Work

2.1 Reranking Recommender System

The notion of reranking in recommender systems has been around since the 90's [6, 23] where researchers have found that the item mutual influence (e.g. diversity) cannot be modeled by the standard learning-to-rank methodology [3, 5], and it has to advance

the optimization problem into the list-wise viewpoint. In other words, the reranking problem is generally formulated as a task that maximizes the list-wise reward given the entire initial list as input, assuming the existence of item mutual influences. Following this setting, several generator-only approaches have been proposed to model and rerank the list as a whole [1, 17, 29, 34, 56]. Some recent work [15, 37] also noticed that large language models [48] can enhance the reranking performance when textual information of items is provided. However, effective and efficient exploration in the permutation space of the recommendation list is challenging, and the ground truth of a reranked list may not be presented in the original dataset, causing a lack of guidance. As a solution to these limitations, the G-E framework has been proposed [26, 39, 41, 49]. On one hand, the evaluation of a list is a much simpler task than list generation, so making the evaluator a final list selector potentially improves the possibility of finding better rerankings. On the other hand, the generator can effectively explore the permutation space through the guidance of the evaluator. Our proposed method further improves this framework in the generator step, where a multi-generator design rather than a single-generator design is adopted to generate high-quality and diverse lists.

2.2 List Generation in Recommendation

There has been a research effort that studies the list-wise modeling in recommender systems. Recently, the research focus has moved from discriminative methods that optimize a list-wise metric [4, 5, 18, 22, 50] into generative methods [2, 26, 29, 30] that model the probabilistic distribution of lists, which are further categorized into autoregressive methods and non-autoregressive methods. The autoregressive approach [2, 29] picks one item at a time during inference, which is favored in language-style scenarios (e.g., users sequentially browse the recommendation list). While the generative model captures the next's relation to the previously selected items, its expected influence on "future" items is captured by the learning objective. As a result, the majority of this type of solution is essentially an energy model that is simultaneously a generator and an evaluator. For reranking tasks, existing works in [2, 17] fall in this category. The non-autoregressive approach [26, 30] directly modifies and outputs the list as a whole, which is well-suited for picture-style scenarios (e.g., presenting a list on one page). While [30] generates all the items in the embedding space, [26] changes the entire list from the initial ranking with trivial actions like swapping. For reranking tasks, existing works in [26, 39] as well as our proposed framework belong to this category.

2.3 Recommendation Diversity

In addition to the ranking accuracy metrics, the diversity metric is also considered critical for recommendation in practice, and there has been a decent amount of research work on diversity modeling from two decades ago to today [6, 9, 10, 14, 27, 42, 52, 55, 57]. The main application of diversity metrics are list-level metrics such as maximal marginal relevance [6], intra-list diversity [57] , α-NDCG [11], and determinantal point process score [9, 16]. Other works also reveal that system-level diversity metrics such as item coverage [19], entropy [36], GINI-index [38], and list variance [30] are also important for maintaining a content-productive environment. In this work, we address the importance of cross-list diversity between different list generators in the MG-E framework (i.e., the list comprehensiveness), which is close to the idea of list variance [30], but different in that the ordering of items is also considered and the metric is evaluated before an evaluator stage for better guidance of list generation.

3 Method

3.1 Problem Formulation

A user request $q \in \mathbb{R}^a$ in the recommendation service encodes the information about the user's current state, which includes the recent interaction history that describes the dynamic preference transition and the static profile features (such as user ID, gender, and location) that identify collective biases in the preference. In the final reranking stage, an ordered list of items $\mathcal{I} = [i_1, i_2, \cdots, i_n]$ is pre-selected by previous stages in the multi-stage recommendation process. The corresponding reranking (i.e., the generator) model $G(q, \mathcal{I})$ takes the user request and the candidate list of items as input and returns a reranked list of items $\mathcal{J} = [j_1, j_2, \cdots, j_n]$ as output. The learning objective is defined as the maximization of the user feedback (i.e., $\max_\theta \mathcal{R}(q, \mathcal{J})$) through reranking for any given user request q, where the reward model could adopt any listwise metric like the aggregation of positive feedback [39] and relevance ranking metrics [1]. During inference of a standard generator-evaluator (G-E) framework [26, 39, 41], the evaluator will estimate all the lists generated by the generator and select the list with the best score as the final recommendation (i.e., Figure 2-a).

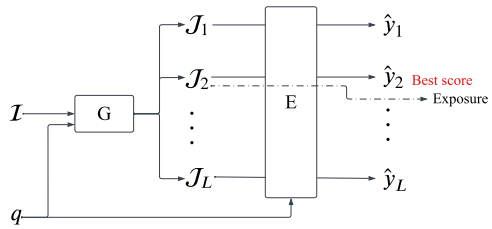
3.2 Multi-Generator Reranking with Evaluator

Our proposed MG-E framework first distinguishes itself from standard G-E by the multi-generator design. As shown in Figure 2-b, the evaluator behaves in the same way as the G-E framework, but M different generator models $G_1, \cdots, G_M$ are included (excluding the $(M+1)$-th extension) and each generator will propose a reranked list to the evaluator. During training, all generators maintain their own training objectives along with the training of the evaluator. In this work, we focus on the design in the generator step and consider the study of the evaluator step as complementary. To keep a fair comparison, we fix the evaluator architecture as the same as the OCPM model in PIER [41] since it has the state-of-the-art ability to model mutual influence between items in a list.

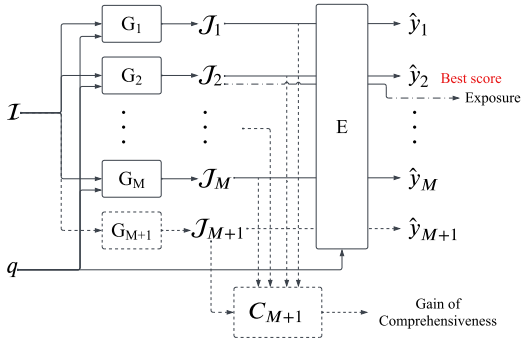
3.2.1 Superiority: In a probabilistic view, each generator G_m determines the generation probability $P(\mathcal{J}|q, \mathcal{I})$ for any permutation $\mathcal{J}$. For deterministic models, only the greedy output list $\mathcal{J}_m$ has probability $P(\mathcal{J}_m|q, \mathcal{I}) = 1$, and all other list permutations assign zero probability. Theoretically, this design forms an ensemble [32, 33] of reranking models, where the overall performance is improved as the number of models increases, for the probability of selecting the optimal list J_* is boosted monotonically with M:

$$P(\mathcal{J}_*|q, \mathcal{I}) = 1 - \prod_{m=1}^{M} (1 - P_m(\mathcal{J}_*|q, \mathcal{I})), \tag{1}$$

where the probability $P_m(\mathcal{J}_*|\cdot)$ denotes the probability of generating the optimal list from the certain generator G_m. We can also



(a) Standard generator-evaluator framework for reranking



(b) The proposed multi-generator-evaluator framework with list comprehensiveness evaluation

Figure 2: Comparison of overviews between standard G-E and the proposed MG-E framework with CLIG extension.

derive a similar conclusion for the b most optimal lists by substituting $P_m(\mathcal{J}_*|q, \mathcal{I})$ with $P_m(\cdot|q, \mathcal{I})$.

3.2.2 Exploration efficiency: Note that the standard G-E framework [41] allows the lone generator to produce more than one list through random permutation, and increasing the number of lists improves the chance of hitting the optimal list. As we have mentioned in section 1, this naive permutation-based list generation is not a cost-efficient exploration strategy under the exponential list permutation space. Specifically, the permutation-based method is effective in finding "similar and high-valued" lists around the local optima, but becomes inefficient in finding "high-valued but strikingly different" lists. In contrast, the multi-generator design is a superior design in solving this limitation by breaking the local optimum restriction (as empirically proved in other generative tasks [21, 24]), and thus improves the list exploration efficiency. This also provides an analytical foundation for our designs of the list comprehensive metric and the complementary list generation technique in the following sections.

3.3 List Comprehensiveness

A good ensemble is one where the individual models are both accurate and behaviorally different [33]. Yet, the straightforward MG-E framework guarantees the generation of accurate lists but not necessarily the list diversity, which potentially reduces the effectiveness of the multi-generator ensemble. As we have illustrated in Figure 1 and section 1, generators may still output the same list for a given q and $\mathcal{I}$, especially when they are trained from the same dataset. We denote this distinctive issue of the MG-E framework as the **list comprehensiveness challenge**.

3.3.1 Metric definition: To better understand this challenge, we first address the importance of the evaluation of the list-level diversity and observe the following two metrics:

- **List comprehensiveness** LC_m evaluates how a given set of lists $\mathcal{J}_1, \cdots, \mathcal{J}_m$ are different from each other:

$$\mathrm{LC}_m = \sum_{m_1, m_2 < m, m_1 \neq m_2} \delta(\mathcal{J}_{m_1}, \mathcal{J}_{m_2}), \tag{2}$$

  where δ denotes the pairwise diversity evaluation kernel. Intuitively, one can improve this metric either by adding a new list or by substituting a list that is more different from other lists.
- **Gain of Comprehensiveness** GC_m evaluates how much a newly added generator G_m contributes to the LC metric:

$$\mathrm{GC}_m = \mathrm{LC}_m - \mathrm{LC}_{m-1} = \sum_{m'=1}^{m-1} \delta(\mathcal{J}_{m'}, \mathcal{J}_m). \tag{3}$$

Note that these metrics estimate cross-list diversity rather than intra-list diversity, so the kernel function δ should represent how different the two given rerankings are.

3.3.2 Choices of list diversity kernel: In practice, when the item embedding is available, one can adopt **embedding-based diversity (EBD)** estimation:

$$\delta(\mathcal{J}_m, \mathcal{J}_{m'}) = 0.5\big(1 - \cos(E(\mathcal{J}_m), E(\mathcal{J}_{m'}))\big), \tag{4}$$

where the embedding function $E(\cdot)$ may use position-weighted pooling of item embeddings when the input is a list, and each item embedding is extracted either through the initial ranker or other pretrained content encoders [8, 53]. Note that we can also regard each list as a cluster and the kernel as cluster diversity measurement, then existing methods like minimal linkage, maximal linkage, and centroid linkage [51] are also feasible. Yet, in cases where the embedding is not available or not in the same unified space across generators, one can adopt a more general kernel function that calculates the total **partial order disparity (POD)**:

$$\delta(\mathcal{J}_m, \mathcal{J}_{m'}) = \sum_{\substack{i,j \in \mathcal{J}_m \\ r(i,\mathcal{J}_m) < r(j,\mathcal{J}_m)}} \mathbb{I}[r(i, \mathcal{J}_{m'}) > r(j, \mathcal{J}_{m'})], \tag{5}$$

where $r(\cdot, \mathcal{J})$ denotes the rank of an item in the ordered list $\mathcal{J}$, and the disparity is larger when the two lists have more disagreement in the partial orders. The metric is minimized when the two lists are identical and maximized when one list is the reverse of the other. Compared to EBD, the POD metric is independent of the embeddings and generator models, but it takes more effort to calculate and is less fine-grained than embedding-based distance measures.

We remind readers that, in addition to the standard $n \rightarrow n$ reranking task, there are other reranking scenarios [39] that include an extra top-k ($k < n$) filtering step after generation. In these circumstances, we can only use POD for the lists (each with size n) before the filtering step, but not the post-filtering lists (each with size k) since the items from different lists may no longer be the same. In contrast, the EBD kernel can effectively estimate both the pre-filtering list (of size n) and the post-filtering list (of size k). We summarize the characteristics of the two kernels in Table 1.

Table 1: Qualitative comparison between EBD and POD diversity kernels for LC metrics.

kernel	allow $n \to k$ filtering	model-agnostic	granularity
EBD	Yes	No	Fine
POD	No	Yes	Coarse

Table 2: Performance comparison between different generator combinations on Avito dataset.

framework	generators	AUC	LC(POD)	LC(EBD)	#list
G-only	G1	0.6876	-	-	1
	G3	0.6911	-	-	1
G-E	G2	0.7066	-	-	1
	PIER-3	0.7180	9.9042	0.2913	3
	PIER-20	0.7185	313.5678	0.6481	20
MG-E	G1+G2	0.7113	8.1562	0.4239	2
	G1+G3	0.7121	9.8750	0.7398	2
	G1+CLIG	0.7135	10.0312	0.7681	2
	G1+G2+G3	0.7225	10.2187	0.7884	3
	+CLIG	0.7297	11.3437	0.8722	4

3.3.3 Empirical study: To further investigate the connection between the LC metric and the ensemble performance, we conduct an offline experiment on the Avito dataset[1] and compare different generator combination strategies. Specifically, we share an OCPM evaluator [41] in the MG-E framework and pick three generator candidates: G1 [12], an item-wise neural model; G2 [39], a permutation-based non-autoregressive model; G3 [2], an autoregressive generative model. We set $n = 5$ for the list size and the permutation space has size $n! = 120$. We summarize the 5-round (random-seeded) average results in Table 2. We can see that increasing the number of generators M generally improves the recommendation performance (i.e., AUC) along with the LC measure, and the three-generator method (i.e., G1+G2+G3) can more effectively boost the performance compared to PIER [41] that yields more than three lists. This verifies the effectiveness of the multi-generator ensemble with different types of generators. Among MG-E methods with two generators, the AUC metric is also positively related to the LC and GC metrics, indicating the importance of choosing generators that provide diverse and comprehensive candidate lists. Yet, this difference also means that a good manual design requires expert knowledge in practice, and we would like to automatically find a generator that can fulfill this task.

3.4 Learning Complementary List Generation

As we have illustrated in section 1, to mitigate the deficiencies of the manually designed MG-E framework, we propose the CLIG extension technique that automatically learns a complementary list generator that can simultaneously align with user preferences and improve the LC metric over the existing generators.

[1]https://www.kaggle.com/c/avito-context-ad-clicks/data

3.4.1 Methodology: Formally, we first assume that there are M existing generators (i.e., $G_1, G_2, \cdots, G_M$) in the MG-E framework, then aim to include a new generator G_{M+1} (with parameter θ) that can simultaneously improve the LC metric and align the reranking with the user preferences, following the notion of finding "high-valued but different" lists. We assume an item-separable reward model [39] (i.e., $\mathcal{R}(q, J) = \sum_{j \in \mathcal{J}} \mathcal{R}(q, j)$). The new generator G_{M+1} determines the list generation probability $P_\theta(\mathcal{J}_{M+1}|q, \mathcal{I})$. The corresponding learning objective for the user preference alignment uses a reward-weighted cross-entropy loss for a generated $\mathcal{J}_{M+1} \sim G_{M+1}(q, \mathcal{I})$:

$$\mathcal{L}_{\text{align}} = -\mathcal{R}(q, \mathcal{J}_{M+1}) \log P_\theta(\mathcal{J}_{M+1}|q, \mathcal{I}) \tag{6}$$

where lists with better rewards tend to further increase the generation probability. For auto-regressive models, we can assign each item-wise reward to the step-wise probability:

$$\mathcal{L}_{\text{align\_ar}} = -\sum_{t=1}^{n} \mathcal{R}(q, j_t) \log P_\theta(j_t|q, \mathcal{I}, j_{0:t-1}) \tag{7}$$

where the auto-regressive model takes the previously selected items $j_{0:t-1}$ as an extra input and j_0 denotes the initial empty list.

Besides, we include the list comprehensiveness enforcement loss that directly uses the GC metric in Eq.(3):

$$\begin{aligned} \mathcal{L}_{\text{comp}} &= -\text{GC}_{M+1} \log P_\theta(\mathcal{J}_{M+1}|q, \mathcal{I}) \\ &= -\Big(\sum_{m'=1}^{M} \delta(\mathcal{J}_{m'}, \mathcal{J}_{M+1}) \Big) \log P_\theta(\mathcal{J}_{M+1}|q, \mathcal{I}) \end{aligned} \tag{8}$$

which encourages the generation of lists with larger gains of comprehensiveness. For autoregressive generators, we adopt the item-wise GC in the modified loss:

$$\mathcal{L}_{\text{comp\_ar}} = -\sum_{t=1}^{n} \Big(\sum_{m'=1}^{M} \delta(\mathcal{J}_{m'}, j_t) \Big) \log P_\theta(j_t|q, \mathcal{I}, j_{0:t-1}) \tag{9}$$

where the diversity kernel δ estimates the difference between an item and a list. In this case, the EBD kernel also makes the corresponding substitution with item embedding, while the POD kernel only considers the partial ordering related to the given item j_t. We provide a detailed description of this alternative in Appendix A.

During training, we combine the two objectives as the final loss function and adopt stochastic gradient descent optimization on θ:

$$\mathcal{L} = \mathcal{L}_{\text{align}} + \lambda \mathcal{L}_{\text{comp}}, \tag{10}$$

where the λ coefficient controls the importance of list comprehensiveness boosting, and the same strategy also applies to the autoregressive models. In the remainder of the paper, we assume the autoregressive model implementation for CLIG due to its superior alignment ability for the sequential recommendation task.

3.4.2 Empirical Study: Using the same experimental setting in Table 2, we compare the two-generator methods with G1 as the existing generator. Compared to G2 and G3, the CLIG extension can more effectively improve the LC metric and further improve the recommendation accuracy compared to manually selected generators. Additionally, according to [30], boosting the cross-list variance (through the LC metric) theoretically guarantees a lower bound on the overall recommendation diversity. We will also provide empirical proof for this connection in section 4.2.

Table 3: Statistics of public datasets in offline experiments

datasets	#items	#users	#requests	positive rate
Avito	23,562,269	1,324,103	53,562,269	3.516%
RecFlow	1,361,856	652,490	24,637,522	60.656%

3.4.3 Including Multiple Extensions. Note that the CLIG technique is an incremental strategy that does not restrict the choice of backbone generators, so the CLIG extension itself can be considered as the backbone and iteratively increases the number of extensions. In section 4.1, we will show that CLIG is a general technique that is effective on different base rerankers, and it can further improve the performance by including more extensions. Intuitively, the combination of different CLIG extensions forms an ensemble that tries to behave differently from each other, similar to the principal component decomposition [25, 47] of data variance.

3.5 Computational Cost and Latency

Compared to single-generator methods, the MG-E framework, either using manual design or CLIG technique, linearly increases the computational cost with the number of generators M. One can avoid the latency overhead by running all generators in parallel, but the computational cost is inevitable. As the number of generators increases, the chance of finding high-value but different lists decreases, so the marginal improvement in recommendation performance theoretically shrinks towards zero as M increases:

$$\begin{aligned} &\lim_{M\to\infty} \left(P_{1:M+1}(\mathcal{J}_*|q,\mathcal{I}) - P_{1:M}(\mathcal{J}_*|q,\mathcal{I})\right) \\ &= \lim_{M\to\infty} P_{M+1}(\mathcal{J}_*|q,\mathcal{I}) \prod_{m=1}^{M} (1 - P_m(\mathcal{J}_*|q,\mathcal{I})) = 0, \end{aligned} \tag{11}$$

Thus, the designer has to balance between the marginally decreasing gain of utility and the linearly increased computational cost in practice. Though not the focus of our paper, we believe that there exists a minimal M that achieves the best balance, and the CLIG extension technique potentially approximates the optimal solution through a greedy generator learning method.

4 Experiments

We summarize the key research questions that require empirical support as follows:

- **RQ1:** Proof of MG-E's superiority in the reranking task, compared with single-generator methods.
- **RQ2:** Proof of the effectiveness of CLIG extension technique that further boosts the MG-E performance.
- **RQ3:** Investigation of LC metrics and GC's effectiveness under different hyperparameters including λ, choices of kernel functions, and the number of extensions.

4.1 Offline Experiments

4.1.1 Experimental Setup.

- **Dataset**: In our experiments, we include two public datasets, Avito and RecFlow. The Avito dataset is a user search log dataset that contains tens of millions of search requests along with the recommendation list and user click responses with $n = 5$, and we follow similar preprocessing steps as PIER [41]. The RecFlow dataset [28] is a recently published multi-stage video recommendation dataset where the reranking task is included as a sub-problem. We adopt an 80-20 split on the dataset according to the timestamp, and for each user's interaction history (with effective_view = 1), we segment it into lists of size $n = 6$ and consider long_view = 1 (i.e., watching sufficiently long time of a video) as positive signals. We summarize the statistics of the two preprocessed datasets in Table 3.
- **Evaluation protocol**: We assume a $n \to n$ reranking scenario and evaluate the AUC and NDCG of the reranked list with user feedback as ground-truth relevance signals. Both metrics are higher when items with higher rewards are reranked in the front of the resulting list. Additionally, to better investigate the correlation between the cross-list diversity and the final recommendation performance, we also include the LC and GC metrics introduced in section 3.3.
- **Baselines**: We include several types of baseline models including a) the item-wise score estimators DNN [12] and DCN [46] that separately learn the user feedback for each user-item interaction; b) Seq2Slate [2] that uses pointer-network to learn the ordering of lists, which is an auto-regressive generator-only solution; c) PRM [34] and EXTR [7] that use transformer architecture to model the mutual influences between items in the list, where we consider them as representatives of non-autoregressive generator-only solutions; d) PRS [13], PIER [41], and NAR4Rec [39] that adopt generator-evaluator framework with single-generator in the first step. Note that the state-of-the-art method PIER allows the single generator to generate multiple lists through permutation-based operations, so we specify the PIER-L alternatives where L represents the number of generated lists. For PIER, while setting the maximum value of $L = n!$ is impractical, we restrict it to the cases with $L = 3$ (compared with the MG-E base method) and $L = 20$ (the original setting in PIER). The NAR4Rec baseline also mentioned the possibility of adding its generator inside an MG-E framework but did not publish the design, so we replicated its implementation according to the paper and considered it as a G-E method.

For all methods, we fix the optimizer to Adam and grid search the learning rate in [0.00001, 0.0001, 0.001, 0.01], the mini-batch size in [256, 512, 1024]. We pick the setting with the best test result as offline validation for later online experiments.

4.1.2 Model Specification: For the proposed MG-E framework, we use the three-generator design mentioned in section 3.3.3 as our base method, which uses an ensemble of DNN, NAR4Rec, and Seq2Slate in the generator step, and OCPM [41] in the evaluator step. During training, we first train the evaluator until convergence, then we separately train each generator. For neural network design, we suggest a transformer-based architecture [34, 43] to better capture the item mutual influences among the items in the initial list and integrate an extra GRU [20] layer if using the autoregressive framework, similar to our model design in the online experiments shown in Figure 4. In practice, we have not found a significant difference between different architectures of reranking models in terms of the final recommendation performance as long as the transformer

Table 4: Recommendation performance comparison in offline experiments. Values in bold represent best results and values with underline represent second best results.

framework	method	Avito		RecFlow	
		AUC	NDCG	AUC	NDCG
G-only	DNN	0.6876	0.6028	0.7067	0.6831
	DCN	0.6896	0.6059	0.7088	0.6856
	Seq2Slate	0.6911	0.6115	0.7098	0.6898
	PRM	0.7124	0.6245	0.7613	0.7004
	EXTR	0.7113	0.6228	0.7641	0.7082
G-E	NAR4Rec	0.7066	0.6205	0.7284	0.6979
	PRS	0.7182	0.6301	0.7639	0.7067
	PIER-3	0.7180	0.6273	0.7640	0.7012
	PIER-20	0.7185	0.6327	0.7645	0.7102
MG-E	Base	<u>0.7225</u>	<u>0.6487</u>	<u>0.7842</u>	<u>0.7217</u>
	+CLIG	**0.7297**	**0.6629**	**0.7878**	**0.7357**

architecture is involved, so we keep our solution description in a model-agnostic manner and focus on the analysis of the learning paradigm. We provide the source code[2] for reproduction.

4.1.3 Main Results (RQ1). We conduct each experiment for five rounds with different random seeds and report the result in table 4. On both datasets, the state-of-the-art G-E methods generally achieve better AUC and NDCG, compared with the generator-only methods, except that the EXTR baseline appears to be better than the (restricted) NAR4Rec baseline and only slightly worse than PIER. In contrast, the proposed MG-E solutions consistently outperform all methods, illustrating the ensemble effect. When generating the same number of lists (i.e. $M = L = 3$), compared with PIER-3, the base model of MG-E improves the AUC by 0.5% on Avito and 2.6% (with student-t significance test value of $p < 0.005$). Additionally, the PIER-20 model increases the number of lists to 20 but only marginally improves the reranking accuracy, while the MG-E continues to be superior to this alternative. These results indicate that the carefully designed MG-E solution can better explore the list permutation space and find better reranking.

4.1.4 Effectiveness of CLIG Extension (RQ2). As shown in Table 4, we can see that the CLIG extension technique can effectively improve the base MG-E model (also statistically significant). This verifies that the GC-based guidance can effectively find the complementary generator and further improve the recommendation performance. To verify the backbone-agnostic nature of CLIG, we integrate the CLIG extension to different G-E backbones and report the results in table 5. We observe that, for all tested backbones, adding the CLIG extension consistently improves the results, verifying its generalizability and flexibility.

4.1.5 Ablation Study (RQ3).

- **Kernel functions:** We compare the two LC kernel functions (i.e., EBD and POD) mentioned in section 3.3.2, and report the result in table 6. We can see that both alternatives improve the recommendation results as long as they describe the list-wise diversity, while the fine-grained guidance with EBD is slightly better than the coarse guidance with POD.
- **Magnitude of $\mathcal{L}_{\text{comp}}$:** We conduct experiments with $\lambda \in [0, 10.0]$ to control the magnitude of GC reward in Eq.(10) when accommodating CLIG on the MG-E base model. We present the results in Figure 3 and show that there exists an optimal point in $\lambda^* \in [0.1, 0.5]$. Intuitively, restricting the λ towards zero would make the extension close to learning an independent generator, which may not guarantee sufficiently diverse list exploration. On the other hand, setting λ to a large value would overwhelm the learning of the original $\mathcal{L}_{\text{align}}$, which negatively impacts the recommendation accuracy. Neither extreme cases achieve better results than the intermediate optimal point.
- **The number of extensions:** As we have illustrated in section 3.4.3, we can iteratively use CLIG to increase the number of generators. We conduct a corresponding experiment that tests different numbers of iterative CLIG extensions and report the result in table 7, which verifies the incremental improvements of including more CLIG extensions.

[2]https://github.com/hellohelen222/LC_MGE_code

Table 5: The comparison result between G-E backbones and their CLIG-extended counterparts.

method	Avito		RecFlow	
	AUC	NDCG	AUC	NDCG
NAR4Rec	0.7066	0.6205	0.7284	0.6979
NAR4Rec + CLIG	0.7154	0.6269	0.7375	0.7011
Improvement	**+1.259%**	**+1.031%**	**+1.249%**	**0.458%**
PRS	0.7182	0.6301	0.7639	0.7067
PRS + CLIG	0.7198	0.6350	0.7698	0.7112
Improvement	**+0.236%**	**+0.777%**	**+0.772%**	**+0.637%**
PIER-20	0.7185	0.6327	0.7645	0.7102
PIER-20 + CLIG	0.7211	0.6468	0.7735	0.7221
Improvement	**+0.362%**	**+2.228%**	**+1.177%**	**+1.675%**
MG-E	0.7225	0.6487	0.7842	0.7217
MG-E + CLIG	0.7297	0.6629	0.7878	0.7357
Improvement	**+0.927%**	**+2.189%**	**+0.459%**	**+1.926%**

Table 6: Comparison results of EBD and POD kernels when adding the CLIG extension on MG-E base model.

method	CLIG w/ EBD	CLIG w/ POD
AUC	0.7878	0.7863
Improvement	**+0.459%**	**+0.268%**

4.2 Online A/B Test

To better investigate how the MG-E framework and the CLIG extension behave in an online environment, we conduct an A/B test on an industrial platform that serves video feeds to hundreds of millions of users on a daily basis. As illustrated in Figure 4, the general workflow of the online system is a multi-stage framework that

Table 7: The comparison result of different number of CLIG extension on MG-E base model.

#CLIG Extension	1	3	5
AUC	0.7878	0.7882	0.7894
Improvement	**+0.459%**	**+0.510%**	**+0.663%**

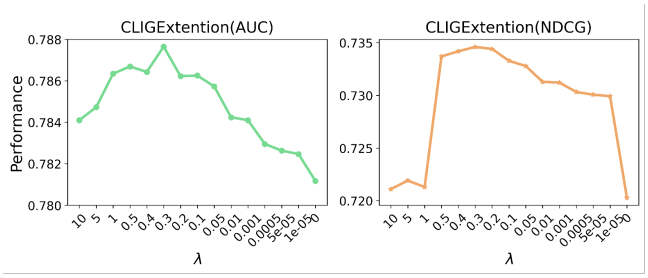


Figure 3: The comparison result of different settings of λ for CLIG extension on MG-E base model.

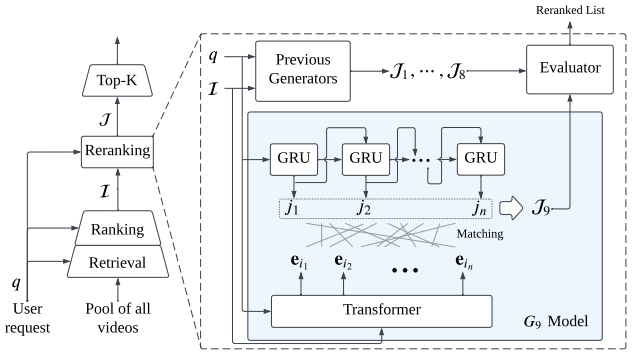


Figure 4: Online recommendation workflow and the detailed forward model of the newly added generator in CLIG.

consists of three stages: retrieval, ranking, and reranking. The last stage conducts a top-k selection after the $n \rightarrow n$ reranking process, where $n = 50$ and $k = 6$. The permutation space near 1.5×10^{10}, which is impractical to enumerate.

4.2.1 Experimental Setup. In our experimental design, the offline evaluation serves as the validation set to choose optimal hyperparameters, while the online A/B testing results are treated as the test set. In turn, the evaluation in the online environment with controlled A/B testing serves as the unbiased test set for reporting performance. This setup ensures a strict separation between validation and test sets, preventing information leakage and overfitting issues during evaluation.

- **Baselines and models**: we use PIER [41] (with $L = 50$) as the baseline and compare it with the MG-E base framework with $M = 3$ manually designed models (which aligns with our setting in the offline experiments). Upon the time we test the CLIG extension, the MG-E framework has progressed to an eight-generator version ($M = 8$). We omit the details of the previous M generators and present a specification of the newly added G_9 model in the EBD-based CLIG extension, as shown in Figure 4. For hyperparameters of CLIG, we set the kernel function to EBD, $\lambda = 0.1$, and restrict to one CLIG extension.
- **Data stream**: For each experiment, we evenly separate the data traffic into ten folds, where two folds (taking 20% of the data stream) are compared as the baseline and we deploy our model alternatives (i.e., MG-E and MG-E with CLIG) in 4 other folds, each taking 2 folds. The data samples are similar to that in the RecFlow dataset, where the users provide various types of feedback including but not limited to watch time, effective view, click like button, sharing, and following the author.
- **Training procedure**: All the aforementioned signals are linearly combined with empirical weights to obtain the item-wise reward during training. Since we adopt an autoregressive model in the CLIG extension, the corresponding learning objective becomes $\mathcal{L} = \mathcal{L}_{\text{align\_ar}} + \lambda \mathcal{L}_{\text{comp\_ar}}$. In online learning, training data continuously evolves along with the model. To avoid catastrophic forgetting [40, 45], we adopt the replay buffer technique that stores recent samples and gives each historical sample a chance of being included in the training at any time.
- **Evaluation protocol**: for accuracy metrics, we directly use the key components in the reward, including the average watch time, effective view count, like count, share count, and follow count. Additionally, to verify the correlation between the CLIG extension and the traditional diversity metrics, we also observe several diversity metrics including category coverage, item coverage, and intra-list diversity [57].

Table 8: Online A/B test result for MG-E. Values with asteroids are statistically significant ($p < 0.05$) results.

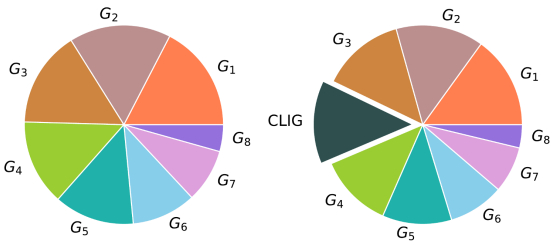
metrics	MG-E(3 model) vs. baseline	avgImpr
watch time	+0.481%*	+0.160%
effective view	+0.416%*	+0.139%
like count	+0.220%*	+0.073%
share count	+0.375%*	+0.125%
follow count	+0.321%*	+0.107%
category coverage	+0.012%	+0.004%

4.2.2 Main Result (RQ1 and RQ2). For both the MG-E validation and the CLIG validation, we conduct corresponding experiments for one week and report the average result in Table 8 and Table 9, respectively. We can see that MG-E outperforms PIER in the reranking accuracy metrics but not necessarily in diversity metrics. Then, adding the CLIG extension can automatically find a complementary generator that significantly improves the diversity and some of the key interaction metrics. This also verifies the limitation of manually designed generators and the generalizability of CLIG.

4.2.3 Generator Selection Bias. To further verify the effectiveness of the CLIG extension, we observe the evaluator's selection frequency of the generators and present the comparison result in Figure 5, corresponding to the comparison in Table 9. We can see that CLIG can rank in the 4th position of all the 9 generators taking relatively 18% of the list exposure, which indicates that it is

Table 9: Online A/B test result for CLIG extension. Values with asteroids are statistically significant ($p < 0.05$) results.

metrics	MG-E + CLIG vs. MG-E
watch time	+0.031%
effective view	+0.308%*
like count	+0.009%
share count	+0.481%*
follow count	+0.309%
item coverage	+11.538%*
intra-list diversity	+11.248%*
category coverage	+0.255%*



a) MG-E b) MG-E + CLIG

Figure 5: Change of list selection bias when adding CLIG.

accurate and sufficiently different from others. We also observe that other generators' exposure scales down at relatively the same rate, indicating an evenly distributed affection, which also validates the CLIG's intuition of being different from other generators. In addition to the ever-changing model parameters, we remind readers that this generator selection bias also affects the online learning data stream, while adding a new generator would take away other generators' chances to contribute to the data stream for certain types of requests. To circumvent this selection bias, we adopt an ϵ-greedy strategy that allows the evaluator to randomly select a candidate list as the recommendation with a small probability ϵ.

5 Conclusion

In this work, we illustrate that the multi-generator-evaluator framework takes advantage of the ensemble effect and consistently outperforms state-of-the-art single-generator models (with or without evaluator). Additionally, we verify that the core list-wise diversity measure to the ensemble effect can be achieved by the list comprehensiveness metric, and derive an automatic generator finding method, CLIG. The CLIG technique is generalizable to different backbone generators and can iteratively increase the number of generators to further improve the performance. In practice, one should balance the computational cost of the included generators and the decreasing improvement of the recommendation performance. Yet, coming with the autonomous generator finding strategy is the unknown inference logic behind each reranking decision, which may require further investigation, especially in scenarios where the reasons for recommendations are important.

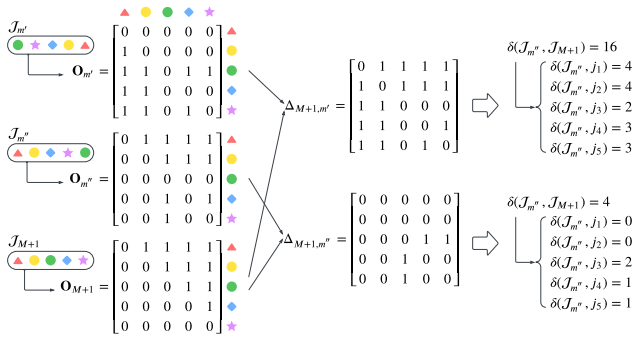


Figure 6: Example of partial order matrix computation of POD kernel. The new $M+1$-th list is compared with two other lists and the list is closer to $\mathcal{J}_{m''}$ compared to $\mathcal{J}_{m'}$.

6 Appendix

A Auto-regressive Kernel

When calculating the GC loss Eq.(9) for autoregressive models, we suggest using the item-wise metric $\delta(\mathcal{J}_{m'}, j_t)$ rather than the standard list-wise GC metric $\delta(\mathcal{J}_{m'}, \mathcal{J}_{M+1})$ as the guidance to each $P_\theta(j_t|\cdot)$, since items may contribute different magnitudes of ranking diversities. For the EBD kernel, we can simply set $\delta(\mathcal{J}_{m'}, j_t) = 0.5(1 - cos(E(\mathcal{J}_{m'}), E(j_t)))$. For the POD kernel, we can adopt a similar idea and assign the conflicting partial orders related to j_t as the item-wise guidance:

$$\delta(\mathcal{J}_{m'}, j_t) = \sum_{i \in \mathcal{I}} \mathbb{I}\Big[\Big(r(i, \mathcal{J}_{m'}) - r(j_t, \mathcal{J}_{m'})\Big)\Big(r(i, \mathcal{J}_{M+1}) - r(j_t, \mathcal{J}_{M+1})\Big) < 0\Big] \tag{12}$$

With the GPU-based parallel computing available, we can efficiently calculate this metric through the partial order matrix formulation. Specifically, we first construct the partial order matrix $\mathbf{O}_m \in \{0,1\}^{n \times n}$ for each list $\mathcal{J}_m$, where $\mathbf{O}_m[i][j] = \mathbb{I}[r(i, \mathcal{J}_m) < r(j, \mathcal{J}_m)]$ represents the order between i and j. One way to efficiently obtain this matrix for a given list is finding the ranks in $\mathcal{J}_m$ for each item in $\mathcal{I}$ as a row vector $\mathbf{r}_m = [r(i_1, \mathcal{J}_m), \cdots, r(i_n, \mathcal{J}_m)]$, then cross-subtract with itself and check if each cell is smaller than zero (i.e., $\mathbf{O}_m = \mathbb{I}[(\mathbf{r}_m^\top - \mathbf{r}_m) < 0]$). Then, we define the matrix subtraction $\Delta_{m',m} = \mathbf{O}_{m'} - \mathbf{O}_m$ and calculate the list-wise kernel for non-autoregressive models (in Eq.(8)) as:

$$\delta(\mathcal{J}_{m'}, \mathcal{J}_{M+1}) = \|\Delta_{M+1,m'}\|_1 \tag{13}$$

and the item-wise kernel for autoregressive models (in Eq.(9)) as:

$$\delta(\mathcal{J}_{m'}, j_t) = \|\Delta_{M+1,m'}[j_t][:]\|_1 \tag{14}$$

We present the details of the process through a detailed example in Figure 6. Additionally, the item-wise POD kernel can be considered as the decomposition of the list-wise kernel, i.e., $\|\Delta_{M+1,m'}\|_1 = \sum_{j_t} \|\Delta_{M+1,m'}[j_t][:]\|_1$, which provides a better alignment between the loss of autoregressive and non-autoregressive models. In contrast, the nonlinear cosine operation does not possess this property, so one may observe significantly different behaviors between Eq.(8) and Eq.(9) using the EBD kernel.

References

[1] Qingyao Ai, Keping Bi, Jiafeng Guo, and W. Bruce Croft. 2018. Learning a Deep Listwise Context Model for Ranking Refinement. In *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval* (Ann Arbor, MI, USA) *(SIGIR '18)*. Association for Computing Machinery, New York, NY, USA, 135–144. https://doi.org/10.1145/3209978.3209985

[2] Irwan Bello, Sayali Kulkarni, Sagar Jain, Craig Boutilier, Ed Chi, Elad Eban, Xiyang Luo, Alan Mackey, and Ofer Meshi. 2018. Seq2Slate: Re-ranking and slate optimization with RNNs. *arXiv preprint arXiv:1810.02019* (2018).

[3] Chris Burges, Tal Shaked, Erin Renshaw, Ari Lazier, Matt Deeds, Nicole Hamilton, and Greg Hullender. 2005. Learning to rank using gradient descent. In *Proceedings of the 22nd International Conference on Machine Learning* (Bonn, Germany) *(ICML '05)*. Association for Computing Machinery, New York, NY, USA, 89–96. https://doi.org/10.1145/1102351.1102363

[4] Christopher JC Burges. 2010. From ranknet to lambdarank to lambdamart: An overview. *Learning* 11, 23-581 (2010), 81.

[5] Zhe Cao, Tao Qin, Tie-Yan Liu, Ming-Feng Tsai, and Hang Li. 2007. Learning to rank: from pairwise approach to listwise approach. In *Proceedings of the 24th International Conference on Machine Learning* (Corvalis, Oregon, USA) *(ICML '07)*. Association for Computing Machinery, New York, NY, USA, 129–136. https://doi.org/10.1145/1273496.1273513

[6] Jaime Carbonell and Jade Goldstein. 1998. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval* (Melbourne, Australia) *(SIGIR '98)*. Association for Computing Machinery, New York, NY, USA, 335–336. https://doi.org/10.1145/290941.291025

[7] Chi Chen, Hui Chen, Kangzhi Zhao, Junsheng Zhou, Li He, Hongbo Deng, Jian Xu, Bo Zheng, Yong Zhang, and Chunxiao Xing. 2022. EXTR: Click-Through Rate Prediction with Externalities in E-Commerce Sponsored Search. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Washington DC, USA) *(KDD '22)*. Association for Computing Machinery, New York, NY, USA, 2732–2740. https://doi.org/10.1145/3534678.3539053

[8] Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. In *Findings of the Association for Computational Linguistics: ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 2318–2335. https://doi.org/10.18653/v1/2024.findings-acl.137

[9] Laming Chen, Guoxin Zhang, and Hanning Zhou. 2018. Fast greedy MAP inference for determinantal point process to improve recommendation diversity. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems* (Montréal, Canada) *(NIPS'18)*. Curran Associates Inc., Red Hook, NY, USA, 5627–5638.

[10] Peizhe Cheng, Shuaiqiang Wang, Jun Ma, Jiankai Sun, and Hui Xiong. 2017. Learning to Recommend Accurate and Diverse Items. In *Proceedings of the 26th International Conference on World Wide Web* (Perth, Australia) *(WWW '17)*. International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 183–192. https://doi.org/10.1145/3038912.3052585

[11] Charles L.A. Clarke, Maheedhar Kolla, Gordon V. Cormack, Olga Vechtomova, Azin Ashkan, Stefan Büttcher, and Ian MacKinnon. 2008. Novelty and diversity in information retrieval evaluation. In *Proceedings of the 31st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval* (Singapore, Singapore) *(SIGIR '08)*. Association for Computing Machinery, New York, NY, USA, 659–666. https://doi.org/10.1145/1390334.1390446

[12] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep Neural Networks for YouTube Recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems* (Boston, Massachusetts, USA) *(RecSys '16)*. Association for Computing Machinery, New York, NY, USA, 191–198. https://doi.org/10.1145/2959100.2959190

[13] Yufei Feng, Yu Gong, Fei Sun, Junfeng Ge, and Wenwu Ou. 2021. Revisit recommender system in the permutation prospective. *arXiv preprint arXiv:2102.12057* (2021).

[14] Zhe Fu, Xi Niu, and Mary Lou Maher. 2023. Deep Learning Models for Serendipity Recommendations: A Survey and New Perspectives. *ACM Comput. Surv.* 56, 1, Article 19 (Aug. 2023), 26 pages. https://doi.org/10.1145/3605145

[15] Jingtong Gao, Bo Chen, Xiangyu Zhao, Weiwen Liu, Xiangyang Li, Yichao Wang, Zijian Zhang, Wanyu Wang, Yuyang Ye, Shanru Lin, et al. 2024. LLM-enhanced Reranking in Recommender Systems. *arXiv preprint arXiv:2406.12433* (2024).

[16] Mike Gartrell, Ulrich Paquet, and Noam Koenigstein. 2016. Bayesian Low-Rank Determinantal Point Processes. In *Proceedings of the 10th ACM Conference on Recommender Systems* (Boston, Massachusetts, USA) *(RecSys '16)*. Association for Computing Machinery, New York, NY, USA, 349–356. https://doi.org/10.1145/2959100.2959178

[17] Xudong Gong, Qinlin Feng, Yuan Zhang, Jiangling Qin, Weijie Ding, Biao Li, Peng Jiang, and Kun Gai. 2022. Real-time Short Video Recommendation on Mobile Devices. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management* (Atlanta, GA, USA) *(CIKM '22)*. Association for Computing Machinery, New York, NY, USA, 3103–3112. https://doi.org/10.1145/3511808.3557065

[18] Yu Gong, Yu Zhu, Lu Duan, Qingwen Liu, Ziyu Guan, Fei Sun, Wenwu Ou, and Kenny Q. Zhu. 2019. Exact-K Recommendation via Maximal Clique Optimization. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (Anchorage, AK, USA) *(KDD '19)*. Association for Computing Machinery, New York, NY, USA, 617–626. https://doi.org/10.1145/3292500.3330832

[19] Nathaniel Good, J. Ben Schafer, Joseph A. Konstan, Al Borchers, Badrul Sarwar, Jon Herlocker, and John Riedl. 1999. Combining collaborative filtering with personal agents for better recommendations. In *Proceedings of the Sixteenth National Conference on Artificial Intelligence and the Eleventh Innovative Applications of Artificial Intelligence Conference Innovative Applications of Artificial Intelligence* (Orlando, Florida, USA) *(AAAI '99/IAAI '99)*. American Association for Artificial Intelligence, USA, 439–446.

[20] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2016. Session-based Recommendations with Recurrent Neural Networks. In *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). http://arxiv.org/abs/1511.06939

[21] Quan Hoang, Tu Dinh Nguyen, Trung Le, and Dinh Phung. 2018. MGAN: Training generative adversarial nets with multiple generators. In *International conference on learning representations*.

[22] Eugene Ie, Vihan Jain, Jing Wang, Sanmit Narvekar, Ritesh Agarwal, Rui Wu, Heng-Tze Cheng, Tushar Chandra, and Craig Boutilier. 2019. SLATEQ: a tractable decomposition for reinforcement learning with recommendation sets. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence* (Macao, China) *(IJCAI'19)*. AAAI Press, 2592–2599.

[23] Thorsten Joachims, Laura Granka, Bing Pan, Helene Hembrooke, and Geri Gay. 2005. Accurately interpreting clickthrough data as implicit feedback. In *Proceedings of the 28th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval* (Salvador, Brazil) *(SIGIR '05)*. Association for Computing Machinery, New York, NY, USA, 154–161. https://doi.org/10.1145/1076034.1076063

[24] Mahyar Khayatkhoei, Ahmed Elgammal, and Maneesh Singh. 2018. Disconnected manifold learning for generative adversarial networks. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems* (Montréal, Canada) *(NIPS'18)*. Curran Associates Inc., Red Hook, NY, USA, 7354–7364.

[25] Ferath Kherif and Adeliya Latypova. 2020. Principal component analysis. In *Machine learning*. Elsevier, 209–225.

[26] Xiao Lin, Xiaokai Chen, Chenyang Wang, Hantao Shu, Linfeng Song, Biao Li, and Peng Jiang. 2024. Discrete Conditional Diffusion for Reranking in Recommendation. In *Companion Proceedings of the ACM Web Conference 2024* (Singapore, Singapore) *(WWW '24)*. Association for Computing Machinery, New York, NY, USA, 161–169. https://doi.org/10.1145/3589335.3648313

[27] Zihan Lin, Hui Wang, Jingshu Mao, Wayne Xin Zhao, Cheng Wang, Peng Jiang, and Ji-Rong Wen. 2022. Feature-aware Diversified Re-ranking with Disentangled Representations for Relevant Recommendation. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Washington DC, USA) *(KDD '22)*. Association for Computing Machinery, New York, NY, USA, 3327–3335. https://doi.org/10.1145/3534678.3539130

[28] Qi Liu, Kai Zheng, Rui Huang, Wuchao Li, Kuo Cai, Yuan Chai, Yanan Niu, Yiqun Hui, Bing Han, Na Mou, Hongning Wang, Wentian Bao, Yun En Yu, Guorui Zhou, Han Li, Yang Song, Defu Lian, and Kun Gai. 2025. RecFlow: An Industrial Full Flow Recommendation Dataset. In *The Thirteenth International Conference on Learning Representations* (Singapore, Singapore) *(WWW '24)*. https://openreview.net/forum?id=vVHc8bGRns

[29] Shuchang Liu, Qingpeng Cai, Zhankui He, Bowen Sun, Julian McAuley, Dong Zheng, Peng Jiang, and Kun Gai. 2023. Generative Flow Network for Listwise Recommendation. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Long Beach, CA, USA) *(KDD '23)*. Association for Computing Machinery, New York, NY, USA, 1524–1534. https://doi.org/10.1145/3580305.3599364

[30] Shuchang Liu, Fei Sun, Yingqiang Ge, Changhua Pei, and Yongfeng Zhang. 2021. Variation Control and Evaluation for Generative Slate Recommendations. In *Proceedings of the Web Conference 2021* (Ljubljana, Slovenia) *(WWW '21)*. Association for Computing Machinery, New York, NY, USA, 436–448. https://doi.org/10.1145/3442381.3449864

[31] Weiwen Liu, Jiarui Qin, Ruiming Tang, and Bo Chen. 2022. Neural Re-ranking for Multi-stage Recommender Systems. In *Proceedings of the 16th ACM Conference on Recommender Systems* (Seattle, WA, USA) *(RecSys '22)*. Association for Computing Machinery, New York, NY, USA, 698–699. https://doi.org/10.1145/3523227.3547369

[32] Ibomoiye Domor Mienye and Yanxia Sun. 2022. A survey of ensemble learning: Concepts, algorithms, applications, and prospects. *IEEE Access* 10 (2022), 99129–99149.

[33] David Opitz and Richard Maclin. 1999. Popular ensemble methods: an empirical study. *J. Artif. Int. Res.* 11, 1 (July 1999), 169–198.

[34] Changhua Pei, Yi Zhang, Yongfeng Zhang, Fei Sun, Xiao Lin, Hanxiao Sun, Jian Wu, Peng Jiang, Junfeng Ge, Wenwu Ou, and Dan Pei. 2019. Personalized re-ranking for recommendation. In *Proceedings of the 13th ACM Conference on Recommender Systems* (Copenhagen, Denmark) *(RecSys '19)*. Association for Computing Machinery, New York, NY, USA, 3–11. https://doi.org/10.1145/3298689.3347000

[35] Jiarui Qin, Jiachen Zhu, Bo Chen, Zhirong Liu, Weiwen Liu, Ruiming Tang, Rui Zhang, Yong Yu, and Weinan Zhang. 2022. RankFlow: Joint Optimization of Multi-Stage Cascade Ranking Systems as Flows. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Madrid, Spain) *(SIGIR '22)*. Association for Computing Machinery, New York, NY, USA, 814–824. https://doi.org/10.1145/3477495.3532050

[36] Lijing Qin and Xiaoyan Zhu. 2013. Promoting diversity in recommendation by entropy regularizer. In *Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence* (Beijing, China) *(IJCAI '13)*. AAAI Press, 2698–2704.

[37] Xubin Ren, Wei Wei, Lianghao Xia, Lixin Su, Suqi Cheng, Junfeng Wang, Dawei Yin, and Chao Huang. 2024. Representation Learning with Large Language Models for Recommendation. In *Proceedings of the ACM Web Conference 2024* (Singapore, Singapore) *(WWW '24)*. Association for Computing Machinery, New York, NY, USA, 3464–3475. https://doi.org/10.1145/3589334.3645458

[38] Yi Ren, Xiao Han, Xu Zhao, Shenzheng Zhang, and Yan Zhang. 2023. Slate-Aware Ranking for Recommendation. In *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining* (Singapore, Singapore) *(WSDM '23)*. Association for Computing Machinery, New York, NY, USA, 499–507. https://doi.org/10.1145/3539597.3570380

[39] Yuxin Ren, Qiya Yang, Yichun Wu, Wei Xu, Yalong Wang, and Zhiqiang Zhang. 2024. Non-autoregressive Generative Models for Reranking Recommendation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Barcelona, Spain) *(KDD '24)*. Association for Computing Machinery, New York, NY, USA, 5625–5634. https://doi.org/10.1145/3637528.3671645

[40] Anthony Robins. 1995. Catastrophic forgetting, rehearsal and pseudorehearsal. *Connection Science* 7, 2 (1995), 123–146.

[41] Xiaowen Shi, Fan Yang, Ze Wang, Xiaoxu Wu, Muzhi Guan, Guogang Liao, Wang Yongkang, Xingxing Wang, and Dong Wang. 2023. PIER: Permutation-Level Interest-Based End-to-End Re-ranking Framework in E-commerce. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Long Beach, CA, USA) *(KDD '23)*. Association for Computing Machinery, New York, NY, USA, 4823–4831. https://doi.org/10.1145/3580305.3599886

[42] Jianing Sun, Wei Guo, Dengcheng Zhang, Yingxue Zhang, Florence Regol, Yaochen Hu, Huifeng Guo, Ruiming Tang, Han Yuan, Xiuqiang He, and Mark Coates. 2020. A Framework for Recommending Accurate and Diverse Items Using Bayesian Graph Convolutional Neural Networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (Virtual Event, CA, USA) *(KDD '20)*. Association for Computing Machinery, New York, NY, USA, 2030–2039. https://doi.org/10.1145/3394486.3403254

[43] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* (Long Beach, California, USA) *(NIPS'17)*. Curran Associates Inc., Red Hook, NY, USA, 6000–6010.

[44] Lidan Wang, Jimmy Lin, and Donald Metzler. 2011. A cascade ranking model for efficient ranked retrieval. In *Proceedings of the 34th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Beijing, China) *(SIGIR '11)*. Association for Computing Machinery, New York, NY, USA, 105–114. https://doi.org/10.1145/2009916.2009934

[45] Liyuan Wang, Xingxing Zhang, Hang Su, and Jun Zhu. 2024. A Comprehensive Survey of Continual Learning: Theory, Method and Application. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46, 8 (2024), 5362–5383. https://doi.org/10.1109/TPAMI.2024.3367329

[46] Ruoxi Wang, Bin Fu, Gang Fu, and Mingliang Wang. 2017. Deep & Cross Network for Ad Click Predictions. In *Proceedings of the ADKDD'17* (Halifax, NS, Canada) *(ADKDD'17)*. Association for Computing Machinery, New York, NY, USA, Article 12, 7 pages. https://doi.org/10.1145/3124749.3124754

[47] Svante Wold, Kim Esbensen, and Paul Geladi. 1987. Principal component analysis. *Chemometrics and intelligent laboratory systems* 2, 1-3 (1987), 37–52.

[48] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, et al. 2024. A survey on large language models for recommendation. *World Wide Web* 27, 5 (2024), 60.

[49] Yunjia Xi, Weiwen Liu, Xinyi Dai, Ruiming Tang, Qing Liu, Weinan Zhang, and Yong Yu. 2024. Utility-Oriented Reranking with Counterfactual Context. *ACM Trans. Knowl. Discov. Data* 18, 8, Article 193 (July 2024), 22 pages. https://doi.org/10.1145/3671004

[50] Fen Xia, Tie-Yan Liu, Jue Wang, Wensheng Zhang, and Hang Li. 2008. Listwise approach to learning to rank: theory and algorithm. In *Proceedings of the 25th International Conference on Machine Learning* (Helsinki, Finland) *(ICML '08)*. Association for Computing Machinery, New York, NY, USA, 1192–1199. https://doi.org/10.1145/1390156.1390306

[51] Rui Xu and D. Wunsch. 2005. Survey of clustering algorithms. *IEEE Transactions on Neural Networks* 16, 3 (2005), 645–678. https://doi.org/10.1109/TNN.2005.845141

[52] Yue Xu, Hao Chen, Zefan Wang, Jianwen Yin, Qijie Shen, Dimin Wang, Feiran Huang, Lixiang Lai, Tao Zhuang, Junfeng Ge, and Xia Hu. 2023. Multi-factor Sequential Re-ranking with Perception-Aware Diversification. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Long Beach, CA, USA) *(KDD '23)*. Association for Computing Machinery, New York, NY, USA, 5327–5337. https://doi.org/10.1145/3580305.3599869

[53] Chao Zhang, Shiwei Wu, Haoxin Zhang, Tong Xu, Yan Gao, Yao Hu, and Enhong Chen. 2024. NoteLLM: A Retrievable Large Language Model for Note Recommendation. In *Companion Proceedings of the ACM Web Conference 2024* (Singapore, Singapore) *(WWW '24)*. Association for Computing Machinery, New York, NY, USA, 170–179. https://doi.org/10.1145/3589335.3648314

[54] Kai Zheng, Haijun Zhao, Rui Huang, Beichuan Zhang, Na Mou, Yanan Niu, Yang Song, Hongning Wang, and Kun Gai. 2024. Full Stage Learning to Rank: A Unified Framework for Multi-Stage Systems. In *Proceedings of the ACM Web Conference 2024* (Singapore, Singapore) *(WWW '24)*. Association for Computing Machinery, New York, NY, USA, 3621–3631. https://doi.org/10.1145/3589334.3645523

[55] Yu Zheng, Chen Gao, Liang Chen, Depeng Jin, and Yong Li. 2021. DGCN: Diversified Recommendation with Graph Convolutional Networks. In *Proceedings of the Web Conference 2021* (Ljubljana, Slovenia) *(WWW '21)*. Association for Computing Machinery, New York, NY, USA, 401–412. https://doi.org/10.1145/3442381.3449835

[56] Tao Zhuang, Wenwu Ou, and Zhirong Wang. 2018. Globally optimized mutual influence aware ranking in e-commerce search. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence* (Stockholm, Sweden) *(IJCAI'18)*. AAAI Press, 3725–3731.

[57] Cai-Nicolas Ziegler, Sean M. McNee, Joseph A. Konstan, and Georg Lausen. 2005. Improving recommendation lists through topic diversification. In *Proceedings of the 14th International Conference on World Wide Web* (Chiba, Japan) *(WWW '05)*. Association for Computing Machinery, New York, NY, USA, 22–32. https://doi.org/10.1145/1060745.1060754